

# Resolución del Práctico 2

## Testing de Software 2024

### Índice

|                                                    |          |
|----------------------------------------------------|----------|
| <b>Introducción</b>                                | <b>3</b> |
| <b>1. Ejercicio 1</b>                              | <b>3</b> |
| <b>2. Ejercicio 2</b>                              | <b>3</b> |
| 2.1. Métodos cubiertos . . . . .                   | 3        |
| 2.2. Archivos agregados . . . . .                  | 3        |
| 2.3. Estrategia de parametrización . . . . .       | 4        |
| 2.4. Ejecución . . . . .                           | 4        |
| 2.5. Resultado esperado . . . . .                  | 4        |
| <b>3. Ejercicio 3</b>                              | <b>5</b> |
| 3.1. Archivos trabajados . . . . .                 | 5        |
| 3.2. Cobertura de pruebas . . . . .                | 5        |
| 3.3. Cambios en la implementación . . . . .        | 5        |
| 3.4. Inconvenientes encontrados . . . . .          | 6        |
| 3.5. Ejecución . . . . .                           | 6        |
| <b>4. Ejercicio 4</b>                              | <b>6</b> |
| 4.1. Archivos . . . . .                            | 6        |
| 4.2. Cobertura de casos . . . . .                  | 7        |
| 4.3. Ejecución . . . . .                           | 7        |
| <b>5. Ejercicio 5</b>                              | <b>7</b> |
| 5.1. Archivos . . . . .                            | 8        |
| 5.2. Cobertura de pruebas . . . . .                | 8        |
| 5.3. Fallas detectadas antes de corregir . . . . . | 8        |
| 5.4. Causa del defecto . . . . .                   | 8        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 5.5. Corrección aplicada . . . . .                        | 8         |
| 5.6. Ejecución y resultado . . . . .                      | 9         |
| <b>6. Ejercicio 6</b>                                     | <b>9</b>  |
| 6.1. Archivos . . . . .                                   | 9         |
| 6.2. Invariante validado por <code>repOK</code> . . . . . | 9         |
| 6.3. Tests parametrizados . . . . .                       | 10        |
| 6.4. Ejecución . . . . .                                  | 10        |
| <b>Conclusiones</b>                                       | <b>11</b> |

# Introducción

Este documento contiene la resolución del *Práctico 2* de *Testing de Software 2024*. El práctico se concentra en el material del capítulo 3 del libro de Ammann y Offutt, en particular en pruebas parametrizadas con JUnit 5 y en el diseño de invariantes de representación (**repOk**). En esa línea, se reescriben como tests parametrizados los casos del Práctico 1, se cubren con pruebas las clases **StackAr**, **Min**, **ZuneBug** y **BoundedQueue**, y se completan los **repOk** pedidos. La continuidad con el práctico anterior es deliberada: muchos de los ejercicios consolidan ideas ya introducidas (como el modelo RIPR) y las extienden hacia formas más sistemáticas de testear.

## 1. Ejercicio 1

Lectura del capítulo 3 del libro *Introduction to Software Testing*.

## 2. Ejercicio 2

El ejercicio pide reescribir como tests parametrizados los casos del ejercicio 3 del Práctico 1. La implementación bajo prueba es la versión defectuosa provista por la cátedra en

- `src/main/java/assignment2_exercises/SimpleRoutines.java`,

de modo que los tests deben fallar exactamente en los casos que exponen cada defecto, replicando el resultado del Práctico 1 pero ahora con una infraestructura más concisa y reutilizable.

### 2.1. Métodos cubiertos

- `findLast(int[] x, int y)`
- `lastZero(int[] x)`
- `countPositive(int[] x)`
- `oddOrPos(int[] x)`

### 2.2. Archivos agregados

- `src/test/java/assignment2_exercises/SimpleRoutinesParameterizedTest.java`
- `src/test/resources/assignment2_exercises/odd_or_pos_cases.csv`

## 2.3. Estrategia de parametrización

Para cumplir con el requisito de usar tanto un método proveedor como entradas en formato CSV, se eligió una fuente distinta para cada método de manera de mostrar las cuatro variantes principales:

- `@CsvSource` en `findLast`, con los pares (`arreglo`, `objetivo`, `esperado`) embebidos en la anotación;
- `@MethodSource` en `lastZero` y `countPositive`, donde un proveedor devuelve un `Stream<Arguments>` con arreglos ya tipados;
- `@CsvFileSource` en `oddOrPos`, leyendo los casos desde un archivo externo (`odd_or_pos_cases.csv`).

La combinación elegida satisface lo pedido y deja en claro el costo-beneficio de cada estrategia: `CsvSource` es liviano para casos simples, `MethodSource` es necesario apenas la entrada deja de ser representable como texto, y `CsvFileSource` se vuelve cómodo cuando los casos crecen en cantidad o se quieren mantener separados del código.

## 2.4. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=
    SimpleRoutinesParameterizedTest test
```

Se incluye `-Djacoco.skip=true` porque la versión de JaCoCo del template (0.8.2) presenta incompatibilidades con las JDK actuales; deshabilitar la cobertura permite que la suite corra sin tocar la configuración del `pom.xml`.

## 2.5. Resultado esperado

La suite ejecuta 10 casos. Sobre la versión defectuosa de `SimpleRoutines` fallan los cuatro que exponen los defectos ya analizados en el Práctico 1:

- `findLast`: no evalúa el índice 0;
- `lastZero`: retorna el primer cero en vez del último;
- `countPositive`: cuenta el 0 como positivo;
- `oddOrPos`: no contabiliza los impares negativos por el comportamiento de `%` en Java.

Los casos restantes pasan, lo que confirma que el conjunto de tests es selectivo: revela los defectos sin generar falsos positivos.

### 3. Ejercicio 3

Se construyó con JUnit 5 una suite de tests que ejercita todos los métodos públicos de `StackAr`, junto con la implementación del método `repOk()`. Los tests utilizan un *fixture* compartido con `@BeforeEach` (una pila de capacidad 3 reiniciada antes de cada caso) y mantienen visibles las fases *arrange*, *act* y *assert*.

#### 3.1. Archivos trabajados

- `src/main/java/assignment2_exercises/stack/StackAr.java`
- `src/test/java/assignment2_exercises/stack/StackArTest.java`

#### 3.2. Cobertura de pruebas

Se cubre el contrato completo de la clase, incluyendo los métodos heredados de `Object`:

- constructor con capacidad inválida (negativa);
- `size`, `isEmpty`, `isFull`;
- `push`, `pop`, `top`;
- `makeEmpty`;
- `equals`, `hashCode`, `toString`;
- `repOk`, tanto en estados válidos como en estados corruptos.

Adicionalmente se incorporaron varios tests negativos para verificar que las precondiciones de la API se respeten:

- `push` sobre pila llena (debe lanzar `IllegalStateException`);
- `push(null)` (debe lanzar `IllegalArgumentException`, en consistencia con el invariante);
- `pop` sobre pila vacía;
- `top` sobre pila vacía.

#### 3.3. Cambios en la implementación

A medida que se completaba el `repOk`, los propios tests revelaron incoherencias que era razonable corregir:

1. Se completó `repOk()` con las condiciones del enunciado: `elems != null`, `sp` dentro del rango `[-1, elems.length)`, ausencia de `null` en la zona ocupada y ausencia de basura por encima de `sp`.
2. Se corrigió `pop()`, que originalmente no devolvía el elemento removido y dejaba la celda desapilada apuntando al objeto. Tras la corrección, `pop` guarda el valor del tope, asigna `null` a `elems[sp]` y recién entonces decrementa `sp`, manteniendo el invariante intacto.
3. Se agregó la validación de `null` en `push`, ya que el invariante exige que la zona ocupada no contenga referencias nulas.

### 3.4. Inconvenientes encontrados

- Probar que `repOk` retorna `false` ante estados internos corruptos (`sp` fuera de rango, huecos en la zona activa, basura más allá del tope) no era posible usando solo la API pública, ya que esa misma API garantiza el invariante. Para forzar esos escenarios se recurrió a *reflection* sobre los campos privados `sp` y `elems`, encapsulando la maniobra en dos *helpers* (`setPrivateIntField` y `getElems`) para que el ruido no se notara en el cuerpo de cada test.
- Implementar `repOk` también ayudó a detectar el bug de `pop` mencionado más arriba: el método rompía el invariante después de cada desapilado, y el chequeo lo dejó en evidencia inmediatamente.

### 3.5. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=StackArTest
    test
```

## 4. Ejercicio 4

Para `Min.min(List<? extends T>)` se diseñó una suite parametrizada que utiliza dos archivos CSV separados: uno con casos válidos y otro con casos esperados de excepción. La elección permite mantener bien identificados los dos tipos de comportamiento sin recargar un único archivo con columnas heterogéneas.

### 4.1. Archivos

- `src/test/java/assignment2_exercises/MinCsvParameterizedTest.java`
- `src/test/resources/assignment2_exercises/min_valid_cases.csv`

- `src/test/resources/assignment2_exercises/min_invalid_cases.csv`

## 4.2. Cobertura de casos

- Casos válidos con enteros, contemplando que el mínimo aparezca al inicio, al medio o al final de la lista, e incluyendo entradas con repetidos, valores negativos y listas unitarias.
- Casos inválidos leídos desde CSV, donde el archivo asocia cada lista de entrada con la excepción esperada:
  - lista vacía → `IllegalArgumentException`;
  - lista con `null` en la primera posición → `NullPointerException` (lo arroja el chequeo explícito tras el primer `next`);
  - lista con `null` en una posición posterior → `NullPointerException` (al invocar `compareTo` sobre `null`).
- Casos negativos adicionales que no caben naturalmente en CSV y se escribieron como tests no parametrizados:
  - lista `null` → `NullPointerException` (al consultar `size`);
  - elementos no mutuamente comparables (mezcla de `Integer` y `String`) → `ClassCastException`.

Para que el CSV pueda expresar listas con `null` y listas vacías se introdujeron dos convenciones: el literal `null` se traduce a `null` en la lista resultante y la palabra clave `EMPTY` representa una lista sin elementos. La conversión de texto a clase de excepción queda concentrada en un único método auxiliar (`exceptionType`), evitando acoplar el archivo CSV al código Java.

## 4.3. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=
    MinCsvParameterizedTest test
```

## 5. Ejercicio 5

Se construyó una suite parametrizada para `ZuneBug.currentYear(int days)` y, a partir de las fallas observadas, se realizó *debugging* y reparación del método. El nombre del ejercicio no es casual: la implementación original reproduce el famoso *Zune bug* del 31 de diciembre de 2008, en el que un manejo incorrecto del año bisiesto dejaba al dispositivo colgado en un bucle.

## 5.1. Archivos

- `src/test/java/assignment2_exercises/ZuneBugParameterizedTest.java`
- `src/main/java/assignment2_exercises/ZuneBug.java`
- `ejercicio5/currentYear_original.java.txt` (versión previa a la corrección).

## 5.2. Cobertura de pruebas

La suite se organiza en dos grupos parametrizados que se complementan:

- un `@MethodSource` con casos de borde elegidos a mano alrededor de fines de año y cambios bisiestos (1, 365, 366, 367, 731, 1461);
- un `@ValueSource` amplio que compara `currentYear` con una función *oracle* basada en `LocalDate`, lo que actúa como prueba diferencial sin necesidad de calcular a mano cada respuesta esperada.

Cada caso se ejecuta dentro de un `assertTimeoutPreemptively` de 200 ms para que un eventual bucle infinito sea detectado como un fallo de timeout en lugar de bloquear toda la suite.

## 5.3. Fallas detectadas antes de corregir

- Con `days = 366`, el método entraba en bucle infinito (era el síntoma exacto del bug original del Zune).
- En transiciones exactas de año — por ejemplo, 731 o 1461 — el resultado quedaba un año por debajo del esperado.

## 5.4. Causa del defecto

La versión original mezclaba dos guardas, `days > 365` en el `while` externo y `days > 366` dentro de la rama bisiesta, sin avanzar el año cuando `days` caía exactamente en el borde. Eso provocaba dos efectos: en años no bisiestos se *quedaba corto* en una unidad, y en años bisiestos con `days = 366` ninguna de las dos condiciones permitía progresar, lo que generaba el bucle infinito.

## 5.5. Corrección aplicada

La idea correcta es decidir, en cada iteración, cuántos días tiene *el año actual* y avanzar sólo si el restante alcanza para cubrirlo entero. La implementación corregida sigue ese esquema:

- se calcula `daysInYear` a partir de `isLeapYear(year)`;
- si `days >= daysInYear`, se descuenta y se incrementa `year`;
- en caso contrario, se rompe el ciclo y se devuelve `year`.

Con esta lógica desaparece tanto el bucle infinito como el *off-by-one* en los bordes, y la verificación contra el *oracle* pasa para todos los valores probados.

## 5.6. Ejecución y resultado

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=
    ZuneBugParameterizedTest test
```

Tras la corrección, la salida reportada es:

- Tests run: 21, Failures: 0, Errors: 0
- BUILD SUCCESS

## 6. Ejercicio 6

El último ejercicio pide implementar `repOK()` en `BoundedQueue` y verificar, mediante tests parametrizados, que el invariante de representación se mantenga después de las operaciones fundamentales de la cola. La estructura es interesante porque, al ser una cola circular, parte del trabajo del invariante es justamente describir cuándo el arreglo subyacente representa una cola legítima.

### 6.1. Archivos

- `src/main/java/assignment2_exercises/queue/BoundedQueue.java`
- `src/test/java/assignment2_exercises/queue/BoundedQueueParameterizedTest.java`

### 6.2. Invariante validado por `repOK`

`repOK` concentra las condiciones que toda instancia bien formada debe satisfacer:

- consistencia estructural básica: `elements` no nulo, `capacity` no negativa y `elements.length == capacity`;
- `size` dentro del rango `[0, capacity]`, con el caso especial `capacity == 0` que exige `size = front = back = 0`;
- `front` y `back` dentro de `[0, capacity)`;

- la relación de cola circular: `back == (front + size) % capacity;`
- ausencia de `null` en la zona ocupada (las `size` celdas a partir de `front`);
- ausencia de *basura* fuera de la zona ocupada: las celdas que no representan elementos vivos deben quedar en `null`.

Las dos últimas condiciones requieren marcar las posiciones ocupadas mediante un arreglo auxiliar `occupied`, dado que el *wrap-around* hace que la zona viva pueda partirse en dos tramos del arreglo.

### 6.3. Tests parametrizados

Los casos se distribuyen en dos `@MethodSource` pensados para cubrir comportamientos contrastantes:

- **Escenarios válidos:** cada caso describe una secuencia textual de operaciones (`E:x` para encolar, `D` para desencolar) sobre una cola de capacidad fija. Para cada operación se verifica `repOK` y, al final, se contrastan tanto el contenido remanente como la lista de elementos extraídos. La elección de capacidades pequeñas (1, 2, 3 y 4) y secuencias mixtas obliga a la cola a hacer *wrap-around*, ejercitando justamente la rama del invariante que valida la zona viva partida en dos tramos.
- **Escenarios inválidos:** agrupan las tres precondiciones explícitas de la API — `deQueue` en vacía, `enQueue(null)` y `enQueue` en llena — y, además de chequear que se lance la excepción correcta, vuelven a evaluar `repOK` para confirmar que el estado interno no se haya corrompido por el error. Esto es importante: una excepción esperada no debería dejar la cola en un estado inconsistente.

La codificación textual de las operaciones (`.E:a;E:b;D"`) facilita escribir muchos escenarios sin repetir el mismo *boilerplate* y mantiene el cuerpo del test centrado en la verificación.

### 6.4. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=
    BoundedQueueParameterizedTest test
```

## Conclusiones

El Práctico 2 retoma el diseño de tests del práctico anterior y lo lleva un paso más allá: en vez de duplicar tests cuando cambian los datos, se aprovecha la maquinaria parametrizada de JUnit 5 para describir familias enteras de casos. En particular, el trabajo permitió:

- ejercitar las cuatro estrategias principales de parametrización (`@CsvSource`, `@CsvFileSource`, `@MethodSource` y `@ValueSource`) y entender cuándo conviene cada una;
- formalizar invariantes de representación en estructuras de datos no triviales — pila acotada y cola circular — y verificarlos mediante `repOk/repOK`;
- reproducir y reparar dos defectos reales (el bug de `pop` en `StackAr` y el *Zune bug* en `ZuneBug`), apoyándose en los tests para guiar el *debugging*;
- combinar pruebas funcionales con validación estructural, dejando un esquema de *testing* mucho más expresivo que el del Práctico 1.